International Journal of Computer Application
Available online on https://rspublication.com/ijca/ijca_index.htm
DOI: 10.5281/zenodo.15844012

ISSN 2250-1797
Issue 15 Volume 4 2025
Original Article

"Kubernetes in AWS (EKS): Enhancing DevOps Workflow Efficiency"

Swarup Panda[1] (✉)
✉ swaruppanda1331@gmail.com

Abstract-

Kubernetes is primarily used for the deployment of the cloud-native application, container-based application, or microservices application built about stateless application architectures. Kubernetes consists of thick client-server architecture, where services run on a cluster of machines, and the user communicates with the K8s master server. Deployments are described in JSON or YAML formats. These deployment descriptors are provided to K8s APIs. Kubernetes opens APIs for both consumers and deployers of services. Kubernetes provides an API abstraction layer to support the deployment of many distributed systems. Kubernetes features rich APIs for controlling various platform features, including application deployment, application scaling, and deployment checking and monitoring.

Keywords- Kubernetes, Aws, DevOps, API , infrastructure as code (IAC), JSON ,YAML

1. Introduction

Cloud services provide the ability for organizations to scale rapidly with no capital expenditure and pay-as-you-go pricing model. The first step in implementing Continuous Integration and Continuous Deployment pipelines in any organization is to automate the build and release process. The implementation of CI/CD pipelines creates multiple merits like the efficient utilization of resources resulting in minimized cost, ability to monitor the build and deploy process from a single solution, faster build and deploy time, readymade pipeline implementations, quick-start provisioned pipelines, and an integrated solution that creates triggers for builds and releases.

2. Understanding Kubernetes

Kubernetes is a leading container orchestration system that queries containers, including building, deploying, and running thousands of containers. It automates the deployment and scaling up/down, management of application containers, and even networks. Kubernetes

Cite This Paper: Swarup Panda (2025). "Kubernetes in AWS (EKS): Enhancing DevOps Workflow Efficiency". *INTERNATIONAL JOURNAL OF COMPUTER APPLICATION (IJCA) ISSN-2250-1797*, vol. 15, no. 4, 2025, pp. 40-63, DOI: https://dx.doi.org/10.5281/zenodo.15844012

ISSN 2250-1797
Issue 15 Volume 4 2025
Original Article

provides a container-centric management environment and offers a rich set of features, including service discovery and load balancing, storage orchestration, automated container deployment, and rollback, self-healing, including auto-replacement, health checking, etc. Kubernetes is developed in the Go language.

2.1. Kubernetes Architecture

The Kubernetes Architecture is a microservices-based architecture that enables the deployment of applications in a distributed manner. In Kubernetes Architecture, a Kubernetes Cluster, which is basically a group of Physical or Virtual Machines, provides the core set of components required to run packages called containers. Within the cluster, the application containers are hosted in at least one of the virtual machines called nodes. These nodes are controlled and managed by components called Control Plane. The Virtual Machines consisting of Kubernetes nodes and Control Plane components can be hosted in any environment, and Kubernetes is not bound to any kind of environment. Kubernetes can be deployed in the Local Environment, Dedicated Data Center, or Hosted Cloud Services. The Components of Kubernetes features a mix of both Server-side and Client-side components. Server-side components are the Components that run on Master Node or Control Plane Node, and the Client-side components are the components that run on Worker Node.

One of the Admin tasks in the Kubernetes cluster is to install and deploy the Pods which in turn are a group of one or more Containers. Kubernetes does the same by using a set of Components running as processes. These Kubernetes Components also monitor the Pods and take any kind of extra steps required to ensure the Pods are not affected by anything and are always Up and Running. The Architecture of Kubernetes provides a very reliable structure to install and manage the enterprise application - both on-premise and hosted services. With such a framework comprising all the essential required services, Kubernetes helps you deploy your application solutions globally with no or less hassle and integration concerns.

2.2. Key Components of Kubernetes

As previously stated, Kubernetes is a complex open-source system and a container orchestration tool. Its rich set of functionalities enables the deployment, management, and scaling of applications in a cloud-agnostic, cloud-provider-neutral, and bare-metal-neutral manner. Kubernetes can not only be deployed in multi-cloud environments but also be used to orchestrate containers across multi-cloud and hybrid-cloud environments. Although Kubernetes was designed to manage and run stateless applications using containers, it can easily be extended to manage and run stateful applications, thanks to the effort put in by many in the open-source community.

Kubernetes is comprised of two main components, the Control Plane, which is the brain or the central nervous system that provides a mechanism to manage the cluster; and the Work

ISSN 2250-1797
Issue 15 Volume 4 2025
Original Article

Nodes, or Data Plane, which provides a mechanism to run containerized applications. Kubernetes exposes a robust set of APIs that enable interactions to deploy applications and report on the health status, from the control plane, as well as to allow the applications deployed to leverage functionality offered by Kubernetes, such as service discovery, from the work nodes. Most of the available Kubernetes distributions, be it on the public cloud or private cloud, follow this two-component architecture with component responsibilities divided in a similar manner.

3. Overview of AWS EKS

3.1. What is AWS EKS?

Amazon Elastic Kubernetes Service is a managed Kubernetes service that simplifies the operation of Kubernetes on AWS without sacrificing your control over the orchestration of containers. AWS takes care of deploying, operating, and scaling Kubernetes control plane infrastructure in your AWS account. EKS is designed for high availability, security, and scalability. EKS directly integrates with other AWS services to launch containers at scale, achieve secure end-to-end networking, and provides integrated CI/CD tooling.

EKS manages the availability of the Kubernetes control plane across multiple AWS Availability Zones for your cluster. You can use customer-managed keys to create and manage your own encryption keys for secrets stored in etcd. By default, EKS uses IAM to provide authentication to the Kubernetes API server. The IAM roles for service accounts feature provided by EKS allows Kubernetes workloads running on your nodes to use IAM roles to access AWS APIs. This feature uses OpenID Connect for authentication, which mitigates the need to put AWS access keys into container images. Control plane communicates with nodes using the Kubernetes API over TLS. The API and etcd are protected using AWS Certificate Manager, which manages the authentication certificates used for encrypting this communication. EKS is fully integrated with the AWS Cloud Development Kit. You can easily define your infrastructure as code and create and manage your EKS clusters via the AWS CDK. EKS uses the Kubernetes add-on feature to manage scalable and highly available CoreDNS and kube-proxy.

3.2. Benefits of Using EKS

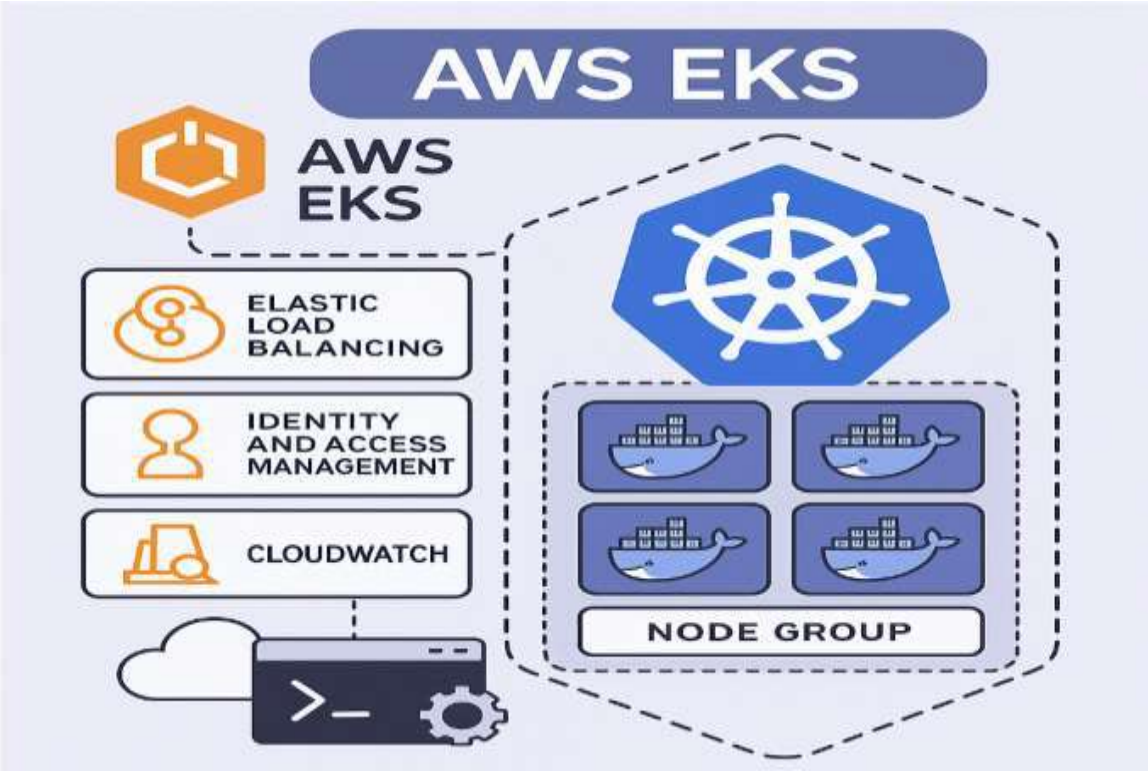
Amazon EKS is available on the AWS Free Tier and integrates seamlessly with existing AWS services. It monitors cluster activity and identifies desired state changes. EKS is deployed across multiple AZs to provide resiliency and high uptime. You can create an EKS cluster in just 6 clicks, which saves time required to create clusters manually. EKS gives authorized users full access to the Kubernetes API to run kubectl commands. It integrates directly with Amazon Inspector to identify services and actions using known vulnerabilities.

ISSN 2250-1797
Issue 15 Volume 4 2025
Original Article

3.1. What is AWS EKS?

Amazon Elastic Kubernetes Service streamlines the provisioning and management of Kubernetes clusters in Amazon Web Services. With EKS, engineers can easily install and run clusters that can scale to meet the needs of business products. EKS integrates other services provided by AWS such as Elastic Load Balancing for distributing traffic on clusters, Identity and Access Management for regulating control of clusters, or CloudWatch for logging logs and generating alerts based on events in the cluster.

AWS EKS is a fully-managed service and its operation is simple: the user can create an EKS-enabled VPC using CloudFormation, configure a Kubernetes control plane using a CLI command, and point it at the newly created managed node group. AWS EKS simplifies cluster setup via node groups, a single resource in charge of managing the shared lifecycle of a group of instances that function as Kubernetes nodes. Node groups facilitate the configuration of instances or groups of instances that are to be used as nodes in the cluster via an Auto Scaling Group. EKS automatically manages the ASG configuration related to the Kubernetes nodes by inspecting the cluster's scaling behavior. The user only needs to provide some parameters to EKS: the Managed Node Group must have a name, EKS must connect that node group to the Kubernetes control plane, and you must choose a minimum and maximum number of nodes for the associated ASG. The user provides the instance types to use, and that information is not user configurable while the node group is active. During the cluster implementation, EKS creates the required IAM policies for the node group.



ISSN 2250-1797
Issue 15 Volume 4 2025
Original Article

3.2. Benefits of Using EKS

Amazon Elastic Kubernetes Service provides a fully managed Kubernetes service that makes it easy for you to run Kubernetes on AWS without needing to install and operate your own Kubernetes control plane or worker nodes. EKS is certified Kubernetes conformant, so existing applications that run on standard Kubernetes can be easily migrated to EKS. EKS runs the Kubernetes management infrastructure for you across multiple AZs to eliminate a single point of failure, and it automatically detects and replaces unhealthy control plane nodes, and provides API access control. EKS also provides automatic scaling and patching for your nodes, built on the enterprise-grade security of AWS.

It can be difficult and time-consuming to manage a Kubernetes cluster. You cannot easily increase or decrease the number of control plane and etcd nodes in your cluster, or replace an unhealthy control plane node. EKS manages your control plane, including the etcd database, which keeps track of the state of your cluster's workloads, for you. Your workloads still run on worker nodes that you manage, including scaling and managing the nodes, but there are a number of benefits to EKS providing the control plane.

Amazon EKS makes it easy to get started with Kubernetes on AWS. Amazon EKS is a fully managed, highly available service that allows you to run Kubernetes without having to install and manage your own Kubernetes control plane. Kubernetes is open-source and community-managed software, and EKS is certified Kubernetes conformant. This means you can use all of the open-source tools and add-ons you are used to, from Kubernetes Dashboard to Helm, with EKS, just as you would with any Kubernetes installation. EKS supports the latest Kubernetes versions, giving you access to the features that matter most. EKS takes care of provisioning, scaling, and managing your control plane across multiple Availability Zones, ensuring that there is no single point of failure. EKS automatically detects and replaces unhealthy control plane nodes, and provides API access control. EKS integrates with services to provide scalability and security for your cluster. And through the Management Console, CLI, and APIs, you have access to controls, monitoring, and management tools for your cluster, built on the reliability of AWS.

4. Setting Up EKS

AWS Elastic Kubernetes Service (EKS) is a managed Kubernetes service designed to simplify deployment and operation of Kubernetes on AWS. EKS handles many clusters' maintenance requirements, such as patching, scaling, and securing control planes. Deploying EKS using the console is a simple and familiar workflow; however, for infrastructure as code (IAC), the easiest way to automate deploying EKS is through AWS Command Line Interface.

ISSN 2250-1797
Issue 15 Volume 4 2025
Original Article

4.1. Prerequisites for EKS

Before creating an EKS cluster, a few steps are required to configure a working environment. EKS relies on Amazon VPC for networking and Amazon IAM for security, so an existing VPC with two private subnets and associated IAM roles must be created. We also need to install or update a few local command-line tools, including the AWS command-line tool, kubectl, eksctl, and AWS IAM Authenticator, before creating a cluster through the CLI. For best practices, it's also recommended to create a new IAM user with policies that grant access to only the commands needed to create a cluster and associated resources.

4.2. Creating an EKS Cluster

Once our local environment is completely configured, the EKS cluster can be created via a single command. eksctl is a tool that automates many of the more tedious tasks required when creating a new Kubernetes cluster. The eksctl command will create both the Kubernetes control plane and the necessary IAM roles that allow the control plane to manage the worker nodes on our behalf. Creating the worker nodes is done either automatically or manually, in that we can specify the configuration desired with the eksctl command. The example below creates the EKS cluster named "dev" and requests 3

ISSN 2250-1797
Issue 15 Volume 4 2025
Original Article

m5.large worker nodes, a type of virtual machine suitable for small workloads like development and testing. Creation will take about 15 minutes, and command output will include information about the IAM console when creation is finished. Command output will also include labels used to create an Amazon Load Balancer for publishing Kubernetes services, which is important for pod configuration.

4.1. Prerequisites for EKS

We need to install and configure AWS Video and kubectl to set our EKS for the first time. We will install and configure AWS Video using Go-based EKS create command and configuration YAML. First of all, we need an accessible AWS account to create an EKS cluster. If you do not have an AWS account, please create one first. After that, you should have an understanding of the programming language that will run in your EKS cluster and minor configuration changes in the cloud and Kubernetes. You should have some knowledge about the networking configuration, such as the CIDR that you want to use for your Kubernetes cluster. You can deploy your application using a web service or a simple internal type service.

Before we continue, you should enable the core infrastructure module in the AWS account using the CLI command. By enabling the core module, the necessary IAM roles, VPC, subnets, and cluster security groups will be created automatically. The important thing is that the VPC CIDR must be bigger than the container CIDR that is configured in the EKS create command. Also, make sure that your account has permissions to create and describe the resources.

If you want to deploy Kubernetes add-ons like CNI, you should set the Kubectl version to the same version as your EKS version, or else it can result in errors during the EKS resource creation or the updating cluster API server session. We will create the resources in the default EKS region, which is “us-west-2”. If you want to use another region, specify the region name in the properties. After that, the Kubeconfig file will be created in the default user home directory. If you want to save the File yaml in another directory, specify the option.

4.2. Creating an EKS Cluster

In this section, we will create the cluster. In EKS, the EC2 instances used for Kubernetes nodes (the compute capacity) are managed by EC2 Auto Scaling Groups. EKS provisions and manages the control plane, but you have to manage the node groups. EKS can use either self-managed node groups (where you create bootstrapped EC2 instances in ASG that can be configured with user-data) or managed node groups which abstract away some of the management actions. In either case, EKS provides pre-configured AMIs for the worker nodes.

ISSN 2250-1797
Issue 15 Volume 4 2025
Original Article

In the following commands, we will combine creating the EKS cluster and the managed node group in one command. You would typically do so for test clusters, while separating them makes sense for production-grade clusters (as you would need the control plane to be provisioned before you can bootstrap the nodes for the control plane to connect to). In production-grade environments, you can also enable logging. The node group will use the latest Amazon Linux based EKS-optimized AMI. You can also create a similar node group with a different operating system but you may face additional steps related to the operating system you choose.

Before we begin, make sure you configure your AWS account CLI to the appropriate account and region where you will provision your EKS cluster. Also note your access credentials for the account as the executed CloudFormation stack will need sufficient access permissions of all the expected managed resource — EKS cluster, managed node group, VPC — for the region/account combination, and be able to provision the resources in a valid state to be able to proceed.

4.3. Configuring kubectl

In this tutorial, we expect that you will follow all the steps to configure kubectl without skipping any. The reason behind this is that there are multiple ways to configure kubectl. Sometimes, installing the configuration via commands does not work due to cache issues, thus in this section, we will show every detail needed to configure kubectl commands for your EKS cluster. It is mentioned that installing aws-iam-authenticator is not needed if we install the AWS CLI version 2. It comes with the presence of the authenticator. When we install the kubeconfig file using the commands, it works for a couple of operations after which we receive errors, thus we need to store it yourself.

There are two parts we need to do to configure kubectl commands: Update the kubeconfig file, and install the aws-iam-authenticator. Also, we will download the kubectl binary if not done already. The kubeconfig file establishes the connection between the Kubernetes API and the AWS IAM. When we issue a command via kubectl, it checks the kubeconfig file, goes to the API, and checks if this user can do that action in the cluster. In the kubeconfig file, the users are not associated directly with the AWS credentials. There is another procedure that needs aws-iam-authenticator, the user, and the corresponding cluster are stated in the kubeconfig file, so during the authentication, that entry is utilized which applies authentication.

5. Integrating DevOps with EKS

EKS offers an enterprise-grade container orchestration service that is managed for you, making it easier to set up a Kubernetes control plane and continue managing the Kubernetes master component. This high availability control plane in AWS allows you to use many of the AWS services, such as virtual private clouds, load balancers, EBS

ISSN 2250-1797
Issue 15 Volume 4 2025
Original Article

volumes, IAM roles, and CloudWatch logs, just to name a few, much more efficiently. However, besides simply creating an EKS Kubernetes cluster, you should also consider how EKS helps you take advantage of the benefits that containers offer for your DevOps processes. Let's take a look at some of the DevOps principles that EKS helps you achieve.

5.1. Continuous Integration and Continuous Deployment

Creating Docker containers and pushing them to a container registry and deploying on Kubernetes clusters has become a very common practice. Kubernetes has a set of controllers that continuously manages and maintains the desired state for the programmed workloads. Kubernetes deployments allow you to create and update your applications in your EKS cluster easily. However, a CI/CD process needs to enable the right conditions for continuous integration and deployment to occur with containers and through EKS. This section will provide an overview of CI/CD and the steps to achieve that with EKS.

Continuous integration is the practice of frequently merging all developer working copies to a shared main branch. The integration process includes building the Docker image and packaging the container images into container registries, always using the proper tags with the SHA or the branch number to maintain consistency and traceability. These actions can be automated by using pipelines, but you have CI/CD services available as a service with various tools.

5.1. Continuous Integration and Continuous Deployment

Continuous Integration (CI) allows developers to ensure that they are integrating their code into shared repositories frequently and continuously while ensuring a high level of quality. It is the practice of continuously merging unsupported code changes into the development repository and triggering automated builds to ensure that the code doesn't break, followed by a series of automated tests that verify the quality of the merged code. Code changes

ISSN 2250-1797
Issue 15 Volume 4 2025
Original Article

trigger a build or are done on a stated interval, often integrated with a test scrape to verify concurrent software quality. With every successful build, the team is informed, and the code that is in the primary development branch is always in a state that can be tested and deployed.

5.2. Infrastructure as Code

Infrastructure as Code (IaC) is one of the top use cases that most organizations adopt on AWS. Infrastructure as code refers to the practice of using code to provision and manage cloud infrastructure resources at scale. Manually creating resources in the console does not allow you to provision and manage cloud resources at scale; it is tedious and it is also difficult to share and audit. Additionally, when you create resources in the console, you cannot easily reproduce the environment. On the other hand, creating and maintaining cloud resources in code can help you version and reuse infrastructure components, use the same templates to refresh environments for dev/test, use the same templates for disaster recovery, and programmatically analyze the security posture of the environment. From the architecture perspective, there is no single place where you can set up all the VPC components with ease. For example, if you're using NLB, then you have to set up the necessary security group rules and other resources with documentation and your partner will have to do that manually, taking a chance that it would not be done correctly. You can use tools to provision and configure the entire VPC with all the necessary components enabling centralized control of the architecture and reducing human error when creating production-ready infrastructure. Moreover, you can use IAM policies to restrict or allow provisioning and configuring certain components based on your organizational needs.

6. Monitoring and Logging

The Kubernetes cluster provisioning and management capabilities elevate the DevOps value proposition, making the approach relevant for an even larger swath of organizations interested in leveraging microservices-based applications at scale. While Kubernetes handles managing the deployment and operational management of a Kubernetes cluster and the underlying compute, networking, and storage resources, the operation of individual containerized workloads deployed to the cluster is still left to the developer. Containerized workload problems — including slow response times, or service unavailability — may become apparent through a variety of logs, debug messages, or alerts, but these may not be sufficient to isolate the root-cause of issues when they happen. This is especially true when there are many resources emitting events, but only a handful are involved in the problem. If you wanted to resolve a problem with an application that you owned, would looking at the log output of every container that has CPU and memory activity help you find out that the cause is an incorrect SQL query in the code of your application?

ISSN 2250-1797
Issue 15 Volume 4 2025
Original Article

6.1. Setting Up Monitoring Tools

Setting up monitoring tools in your EKS cluster will provide significant insights into its health and resource usage. Luckily, there is an easy path for those using Prometheus and/or Grafana by providing a "one-click" solution using the Managed Service for Prometheus and Managed Grafana. This solution not only takes care of all of the configuration of these tools, but also will set up IAM permissions to allow your EKS cluster to send Prometheus metrics to the Prometheus service.

Before you begin, be sure that you complete the prerequisites to ensure a successful deployment. After provisions have been completed, follow the step-by-step instructions for using the Managed Service for Prometheus and Managed Grafana to deploy monitoring tools into your EKS cluster.

As you examine the dashboards in Grafana, you will see an assortment of useful metrics that Prometheus is scraping from the EKS cluster. The presence of these rich metrics gives you additional telemetry into your EKS cluster. As you investigate particular areas of interest, you may want to store, log, audit, or query the telemetry data for future analysis. You may want to analyze small sets of data on an ad hoc basis, or you may want to implement ongoing analysis with rules to detect conditions of interest.

6.2. Logging Best Practices

Managing logs from Microservices is very important in Kubernetes environments. However, logs can grow in size in the Storage layer which makes management a tedious job. Application and Infrastructure log aggregations in an infrastructure using the stack are presented in this chapter.

Implement and Push some conventional approaches. Ingestion, Storage and Management of logs should be adequately segregated from infrastructure components. Storage solutions can be used for Ingestion, and can host Windows/ Linux based log collect agents natively. Send logs directly to an in a cross-account or cross-region environment. The Microservice logs, however, should be separated to build observability. Kubernetes offers an option of scalable, expert-driven solutions using plugins. Cloud providers support scalable solutions as part of their offerings at Economies of Scale. They act on a Service level agreement model where log management becomes affordable. Introduced for Audit use, in the past, solutions became Cloud-Native for their ease of use and development.

Most of log management solutions expect logs to be in the metadata format. Structured logging facilitates avoiding issues related to unknown logs, Search and Querying. Log parsers format streams Info, Warn, Error for better attribution. The Cloud-Native log managers can take external feed as push-based, and pull-based for management of logs from remote sites. The choice of service/partner can be based on cost and Time to Deployment. But easily will set up the Cloud-native-based solution in which is also an

ISSN 2250-1797
Issue 15 Volume 4 2025
Original Article

environment scaling dynamically. Do remember that the service also has some cumulative cost associated with the usage of logs while sending to or the Cloud- Native log manager.

7. Security Best Practices

The AWS EKS service is an abstraction on top of user-managed EC2 clusters that are running the Kubelet and other core components of the Kubernetes architecture. The responsibility of securing the EKS control plane is on Amazon, but user security best practices must be implemented to ensure important assets are not exposed to the world. Here is a list of security best practices that should be utilized when deploying an application onto EKS.

7.1. IAM Roles and Policies

IAM Roles and Policies are essential components of AWS security. An IAM Policy is a document that defines AWS security permissions for an entity, while an IAM Role is an AWS Identity Access Management resource that creates a set of permissions that applications or users can assume to access AWS resources. A service role is a unique type of role that is assumed by an AWS service, such as Amazon EC2. When you create an Amazon EKS cluster, you must associate an IAM role with the cluster to allow Amazon EKS to make calls to the AWS API on your behalf.

ISSN 2250-1797
Issue 15 Volume 4 2025
Original Article

IAM Policies are written in JSON format and describe specific permissions necessary to perform AWS actions on resources. These policies can be attached to IAM users, groups, or roles. AWS recommends following the principle of least privilege when assigning IAM Policies. The principle of least privilege is a best practice in security, and it states that, whenever possible, applications or users should only be granted the permissions required to access the resources needed for performing their jobs. It's wise to enable only those IAM Policies required for the execution role. By default, AWS assumes all principals have no permissions. For example, to Step Functions, an IAM Policy might look like this:

"Effect": "Allow", "Action": "stepfunctions:StartExecution", "Resource": "*" }

To help you keep your IAM Policies secure, AWS IAM generates alerts whenever a policy allows full access to a service. For example, you will receive an alert whenever any principal has this policy applied to it: "Effect": "Allow", "Action": "*", "Resource": "*" }

7.2. Network Security

Network level security is one of the major use cases for the Virtual Private Cloud (VPC) and its associated services. With VPC, you can control network based access to your services, whether it's ECS, APIs, databases, etc. VPCs have several associated features that you will use in varying combinations based on your use case. VPC uses subnets to define additional networking zones within a VPC. Typically, a VPC will have two types of subnets. The first is a public subnet that hosts resources that are publicly reachable via the Internet, such as ECS services running public facing APIs and load balancers for HTTP(S) traffic. The second, private subnet, hosts the rest of your resources such as ECS service instances, databases, etc. A private subnet captures the idea of isolation from the outside world. Resources inside a private subnet cannot be reached directly via the Internet. Traffic sent from these resources to the Internet can, however, be routed via a NAT instance that you create in the public subnet. Resources can communicate with each other within a VPC, and optionally across different VPCs in different accounts. Additionally, a VPN Gateway allows VPN access from a corporate data center into the VPC.

8. Scaling Applications

While scaling of components is partially handled by Service Discovery and Load Balancing, pods corresponding to deployments and daemonsets may need to be scaled to handle workload efficiently, especially when resource requests for CPU and memory are defined properly. Kubernetes provides two types of tools for scaling: Horizontal Pod Autoscaling for deployments and daemonsets, and Cluster Autoscaler for nodes.

Horizontal Pod Autoscaling

Horizontal Pod Autoscaler automatically scales the number of pods in a replication controller, deployment, or replica set based on observed or predicted CPU utilization (or,

ISSN 2250-1797
Issue 15 Volume 4 2025
Original Article

with the Custom Metrics support, on some other, application-provided metrics). HPA periodically queries each of your components' metrics and calculations, and scales the components to maintain a target level of those metrics.

Configuring HPA is very easy as long as the target metrics and the metrics server are correctly set. Metrics can be exported by the application itself or collected through exporters. To create a scaling rule based on CPU utilization of a pod, we can use the following command:

Once created, we can check the autoscaling status with the following command:

It is also possible to create HPA using a manifest file. The supported metric sources are: Object metrics such as a per-pod metric from External Metrics, Pods metrics such as the average CPU utilization across all pods or the average memory utilization across all pods, and Resource metrics such as the average CPU utilization across all pods or the average memory utilization across all pods.

8.1. Horizontal Pod Autoscaling

Horizontal Pod Autoscaling allows Kubernetes to automatically scale the number of pods in a deployment depending on CPU and memory load. Scaling of application workloads is required for applications with fluctuating loads throughout a day or week. In several situations, user load may increase well beyond expected values. For example, a company may send a marketing email to its entire customer base, causing a sudden surge of user activity to the company's web application, leading to performance degradation. Additionally, web applications operate in different time zones across the world, leading to an inconsistent load throughout the day. Continuous 24x7 operation and cost are also an important factor. Scale up pods to meet the required SLA only when needed for an application. For all the other times, keep the minimum pods running to save on the cost.

8.2. Cluster Autoscaler

A cluster has a minimum size and a maximum size specified in the configuration with minimum and maximum number of nodes. The Cluster Autoscaler ensures that the cluster size cannot exceed the maximum specified size and if the application is not running, then the cluster will scale down the nodes to the minimum specified size. If the autoscaler tries to decrease the number of nodes in a cluster and jobs are still pending on those nodes, the autoscaler will not reduce the nodes and issue a warning. When the resources needed for the jobs exceed the capacity of the cluster, then the cluster will be in a pending state for scaling the cluster size and running the jobs within the cluster.

9. Cost Management in EKS

Managing costs in Amazon EKS Infrastructure is an essential component of efficient DevOps Activities. Based on microservices architecture, organizations have varying needs

ISSN 2250-1797
Issue 15 Volume 4 2025
Original Article

in terms of infrastructure resources on Kubernetes. EKS is a fully managed service that handles Kubernetes control plane while you take care of the rest of the infrastructure. You only pay for the EKS cluster and Kubernetes worker nodes. EKS tries to keep things simple by charging a flat fee for the service, irrespective of how many resources you are consuming on the cluster. The worker nodes have to be EC2 instances running on either On-Demand or Spot pricing model and running the cluster. Adding to this is the cost incurred for the..

Understanding EKS Pricing

Pricing Amazon EKS is straightforward, and there are two main components for which you will incur charges. EKS is an OpEx model. You pay for what you use but careful management of cluster size and node utilization is required to avoid excessive costs. The first is the EKS control plane, which you pay at a fixed hourly rate regardless of how many worker nodes or pods you have running in the cluster at any given time. The second is the EC2 instances you use as nodes in your cluster. Pricing follows standard EC2 instance pricing, typically billed at an hourly rate based on the instance type. You will incur additional charges depending on the type of node instances you select, including EBS transactions, network transfer, or if you use other relevant add-on features in EKS.

9.1. Understanding EKS Pricing

A growing number of organizations are using Amazon Elastic Kubernetes Service to deploy and manage Kubernetes in an AWS cloud native architecture. Amazon EKS is a "managed" Kubernetes service that offloads routine management and maintenance tasks from your DevOps team, relieving them to focus on operations and development. While EKS greatly reduces the management overhead of using Kubernetes, there is still a cost to using Amazon EKS. In this section we will take a look at the pricing of EKS to ensure your organization understands what you are getting billed for.

9.2. Cost Optimization Strategies

Modern cloud infrastructure is complex, decentralized, and relatively opaque, comprising many interconnected resources across multiple services. Each service may consist of a large number of assets and, in addition, any of these assets may simultaneously serve a mixture of varying purposes. It is not always clear or trivial how to accurately assess a specific resource’s usage. It is also the case that cloud infrastructure typically incurs unexpected costs, such as ingress or egress traffic spending, third-party integrations, and operations completely outside of specific services.

Firstly, understand how compute resources are billed. The strategy is not just minimizing usage itself but knowing what the actual compute load is. While most workloads don’t change often, there are often large increases or decreases in total load during peaks of development or staging, and during the time in which releases are rolled out. Bursting can

ISSN 2250-1797
Issue 15 Volume 4 2025
Original Article

also occur during company crises or endpoint failures. Also note that compute services are not all charged at the same price; spot nodes can be mixed even into a standard cluster, but removing refresh delays may mitigate their price advantage against other nodes.

10. Troubleshooting Common Issues

While Kubernetes proposes a uniform model for running containers in production, the whole reality isn't so simple. The typical microservices architecture and other important decisions have consequences that can significantly complicate troubleshooting. Thanks to the Kubernetes API, the abstraction provided by Kubernetes for containers running in clusters, and the high level of automation introduced in application deployments by tools, common troubleshooting can be done in a cluster without SSHing into nodes anymore in most cases. This is particularly true for cloud-based managed Kubernetes distributions.

10.1. Identifying Cluster Issues

Applications deployed on a Kubernetes cluster run in isolated environments (pods). Kubernetes injects services into the cluster, which may invoke some operations that may fail. Networking components and firewalls may block some of these invocations. The services that interact with your application services are exposed to the public via a load balancer. Only those operations that connect to the databases for which connections have been flushed out to their logs must show errors. However, other services within a cluster can attempt to connect to the database. Monitoring such log statements in the logs of application services would help identify operations or services that are failing. Only those log outputs that are related to authentications of the application pods connecting to database pods would reflect the database connection issues.

10.2. Debugging Applications

To debug applications running on EKS, we can use CloudWatch, load balancing service, Kubernetes events, and other Kubernetes commands to get the wide range of logging and debugging information possible. There are many ways to log messages into CloudWatch when running applications in EKS, which varies based on the EC2 and Fargate containers we are running. For EC2, we can install the CloudWatch agent manually or deploy the agent using the Quick Start Launch Wizard. For Fargate, we will deploy the Fluent Bit DaemonSet, which will gather container logs from the stdout and stderr streams sent to CloudWatch. It is important to note that the CloudWatch agent only supports EC2 workloads.

In addition to logging messages, there are other commands that can be executed to converse with the applications when troubleshooting. The kubectl describe command provides information for both EKS resources and Kubernetes objects. This command provides much more contextual information than the kubectl get command, including error messages. A common use of this command is to check the service information, including the ClusterIP

ISSN 2250-1797
Issue 15 Volume 4 2025
Original Article

address and the external load balancer address. Essentially we would execute the command to get more information on the Kubernetes service providing the external load balancer.

11. Case Studies

Real-world implementations provide insightful perspectives on the practical use of Kubernetes as a central component of the infrastructure underpinning CI/CD pipelines. Case Study I presents aspects of a successful enterprise project, while Case Study II abstractly outlines a project in the related area of CI/CD decision management, not directly related to Kubernetes. These case studies highlight lessons learned regarding the DevOps workflow, CI/CD implementation, and adopted practices that were challenged once implemented in production.

11.1. Successful Implementations

K2-Bridge-K and the CloudBees CI pipeline are two Kubernetes-related projects implemented by CloudBees, Inc. and Infinera Corporation. After some years of evolution, the CloudBees CI Kubernetes Plugin became the K2-Bridge-K project, which has been used in production by 89 organizations, performing over 51 million builds and 7 million deployments every month. K2-Bridge-K aims to orchestrate native Kubernetes build and deploy workflows in a hybrid mode. That is, Kubernetes orchestrates build operations that are executed by Jenkins agents deployed on non-Kubernetes worker nodes, while Kubernetes orchestrates deploy operations executed by Jenkins agents deployed on Kubernetes worker nodes.

Infinera Corporation is horizontally scaling and speeding the throughput of CI/CD operations by using the CloudBees CI pipeline that simultaneously builds and deploys more than 300 microservice applications that incrementally consume updates from source control management and component repositories. The key enablers of the project pipeline are the Kubernetes API, and native and open-source Kubernetes technologies without any proprietary abstractions. The pipeline successfully uses in the build phase the open-source project, which is optimized.

11.1. Successful Implementations

Chapter Summary The goal of this chapter is not to give theoretical lessons on what DevOps or DataOps are. Our goal instead is to provide practical insights, detailing a few specific use cases from some experienced companies, in the hope to provide useful tips and inspirations for your own company. This chapter illustrates successful Adaptive Data Automation implementations involving real customers and solutions. Each case study includes Customer background information including size, industry, structure, and challenges addressed, Solution description, and Key Benefits for the customer. 11.1. Successful Implementations Intercloud is an Interconnect Fabric as a service that offers

ISSN 2250-1797
Issue 15 Volume 4 2025
Original Article

large Hosted Private Clouds, utilizing Microsoft Azure Stack. Part of the Intercloud offer is enabling large customers to make cost-effective internal digital transformations. Intercloud was increasingly asked to deal with customers' slow internal transformations, getting into elaborate and time-consuming technical discussions. This slowed down Intercloud's own growth in its digital transformation. Solution Description Adapting Lean and Agile Development Management techniques and defining steps for automation for callable microservices, hosted on Microsoft Azure, allowed for building simple, cheap, and efficient data pipelines utilizing Azure Data Factory. With responsibility aligned along business lines, products are now being released frequently, websites and API stability and performance are being monitored, and service metrics defined and trained for triggering alerts and managing escalations in case of trouble. In the meantime, Intercloud has streamlined its processes, defining clear customer roadmaps, speeding up the service documentation and its account management systems. With Adaptive Data Automation in place, the services are now monitored and changing according to business needs. Key Customers Benefits The vertical domains define their services and are responsible for consumption-based service metrics. Data Engineering corners are being set up, and budgets agreed to accelerate key business processes and core competency development. These actions focused on automation provide customers the infrastructure and processes to successfully accelerate their Digital Transformations.

11.2. Lessons Learned

The application for DevOps on EKS has provided several advantages. Snapshots creation has facilitated the ability to test patching and DPA. AD and ADI agent deployment is simplified and, as a requirement, secure in the Air-Gapped environment. Business recovery has also been simplified. Rescale/Clone has succeeded in deploying add and reserve/scale and in business resiliency.

12. Future Trends in Kubernetes and EKS

Besides its popularization in the current context of the modern world related to Cloud Computing, efficiency, and speed of becoming a product, Kubernetes and, consequently, Kubernetes on AWS will still evolve in several aspects of use by DevOps professionals or teams. This chapter describes some aspects that are to become trends on increasing on demand by companies of their DevOps workflow processes.

12.1. Emerging Technologies

Among the trends in Kubernetes, you can expect native serverless Kubernetes support. In addition to the adoption of admission controls, policy engines, automated admission, lessons learned from serverless computing, native integration with WebAssembly and WASI, serverless Kubernetes, etc. In terms of Sidecars, Sidecar containers enable the use of specialized logging, monitoring, security, and networking proxies. Everything as a

ISSN 2250-1797
Issue 15 Volume 4 2025
Original Article

service also comes to Kubernetes: Cluster as a service, Server as a service, Traffic management as a service. Kubernetes service meshes around various technologies will offer sophisticated Layer7 traffic management. Kubernetes is expected to offer adapter as a service for legacy integration challenges.

In the IoT context, it is also expected that leveraging container technologies will enable faster IoT device training iterations and Local Kubernetes Clusters—low-cost local hardware running microK8s can enable faster iterations of localized applications—providing fast deployment connecting the use of function mesh actively enabling the Kubernetes Process, Productivity, and Portability. On the Edge Computing, the adoption of Kubernetes on the Edge has more demand from enterprises as IoT growth continues to accelerate, and there is increased practicality of border mobile data centers and Kubernetes on-boarding and easy remote management. Sure, there are still a lot of things on the evolution of DevOps concerning Kubernetes. Before that actual context of evolution, Kubernetes slowed down because its proposition is really that implemented DevOps has a product-oriented approach, scaling hours of development. However, Kubernetes is not mature enough in terms of being a straightforward solution to manage.

12.1. Emerging Technologies

Over the years, the realm of technology has witnessed an outpouring of innovative technologies. What once lived out in the imagination is today a reality. Underneath this veil of advanced technologies and inventions lurks a foundation of emerging technologies that are developing at a rapid pace. Emerging technologies can be defined as cutting-edge technologies whose development, practical applications, or both are still largely unrealized. For many companies, consumer products, or technology enthusiasts - investing in emerging technologies can have its advantages.

Immense transformations in the world occur when significant advances and breakthroughs in technology are made. Over the next few years, rapid developments relating to Artificial Intelligence, Machine Learning, Cybersecurity, and Cloud Computing are expected to present untapped possibilities. Emerging technologies are categorized under four major types of technology - Specific Purpose Technologies, General Purpose Technologies, Enabling Technologies, and Technology Modification. Some of the most influential and promising advanced technologies that are breaking new ground involve Artificial Intelligence, Blockchain, Cloud Computing, Cybersecurity, Data Analytics, Internet of Things, Photonic Computing, Quantum Computing, Robotic Process Automation, and Digital Twin. Emerging technologies support a multitude of business transformation initiatives, such as Industrial Internet of Things development, Digital Twin implementation, and Artificial Intelligence infrastructure, among others.

ISSN 2250-1797
Issue 15 Volume 4 2025
Original Article

References:

1. Chantry, M., Christensen, H., Dueben, P., & Palmer, T. (2021). Opportunities and challenges for machine learning in weather and climate modelling: hard, medium and soft AI. *Philosophical Transactions of the Royal Society A*, *379*(2194), 20200083.

2. Deser, C., Lehner, F., Rodgers, K. B., Ault, T., Delworth, T. L., DiNezio, P. N., ... & Ting, M. (2020). Insights from Earth system model initial-condition large ensembles and future prospects. *Nature Climate Change*, *10*(4), 277-286.

3. Kadow, C., Hall, D. M., & Ulbrich, U. (2020). Artificial intelligence reconstructs missing climate information. *Nature Geoscience*, *13*(6), 408-413.

4 Rath, Tapan & Panda, Sibaram. (2025). RPA to Automate ETL Processes in Data Warehousing. International Journal of Science and Research (IJSR). 1466-1473. 10.21275/SR25523162324.

5. Barnes, E. A., Toms, B., Hurrell, J. W., Ebert-Uphoff, I., Anderson, C., & Anderson, D. (2020). Indicator patterns of forced change learned by an artificial neural network. *Journal of Advances in Modeling Earth Systems*, *12*(9), e2020MS002195.

6. Irrgang, C., Boers, N., Sonnewald, M., Barnes, E. A., Kadow, C., Staneva, J., & Saynisch-Wagner, J. (2021). Towards neural Earth system modelling by integrating artificial intelligence in Earth system science. *Nature Machine Intelligence*, *3*(8), 667-674.

7. Mahesh, B. (2020). Machine learning algorithms-a review. *International Journal of Science and Research (IJSR).[Internet]*, *9*(1), 381-386.

8. Zhou, Z. H. (2021). *Machine learning*. Springer nature.

9. Alpaydin, E. (2021). *Machine learning*. MIT press.

10. Jordan, M. I., & Mitchell, T. M. (2015). Machine learning: Trends, perspectives, and prospects. *Science*, *349*(6245), 255-260.

11.Mitchell, T. M., & Mitchell, T. M. (1997). *Machine learning* (Vol. 1, No. 9). New York: McGraw-hill.

12. Hacker, P., Engel, A., & Mauer, M. (2023, June). Regulating ChatGPT and other large generative AI models. In *Proceedings of the 2023 ACM conference on fairness, accountability, and transparency* (pp. 1112-1123).

13. Wu, T. Y., Majeed, A., & Kuo, K. N. (2010). An overview of the healthcare system in Taiwan. *London journal of primary care*, *3*(2), 115-119.

14. Yenduri, G., Ramalingam, M., Selvi, G. C., Supriya, Y., Srivastava, G., Maddikunta, P. K. R., ... & Gadekallu, T. R. (2024). Gpt (generative pre-trained transformer)–a comprehensive review on enabling technologies, potential applications, emerging challenges, and future directions. *IEEE Access*.

ISSN 2250-1797
Issue 15 Volume 4 2025
Original Article

15. Panda, Sibaram Prasad. "AI in Decision Support Systems." International Journal of Advanced Multidisciplinary Research, Cases and Practices, 2021. doi:10.5281/zenodo.15448128.

16. Floridi, L. (2023). AI as agency without intelligence: On ChatGPT, large language models, and other generative models. *Philosophy & technology*, *36*(1), 15.

17. Osisanwo, F. Y., Akinsola, J. E. T., Awodele, O., Hinmikaiye, J. O., Olakanmi, O., & Akinjobi, J. (2017). Supervised machine learning algorithms: classification and comparison. *International Journal of Computer Trends and Technology (IJCTT)*, *48*(3), 128-138.

18. El Naqa, I., & Murphy, M. J. (2015). What is machine learning?. In *Machine learning in radiation oncology: theory and applications* (pp. 3-11). Cham: Springer International Publishing.

19. Rath, Tapan & Panda, Sibaram. (2025). Infrastructure as Code (IaC) Evolution - A Comparative Study of Terraform and AWS Cloud Formation. International Journal of Science and Research (IJSR). 1476-1483. 10.21275/SR25523162246.

20. Panda, Sibaram Prasad. "Real-Time Monitoring and Feedback Loops in DevOps Using Azure Monitor and Log Analytics." International Journal of Advanced Multidisciplinary Research, Cases and Practices, 2022. doi:10.5281/zenodo.15448657.

21. Burrell, J. (2016). How the machine 'thinks': Understanding opacity in machine learning algorithms. *Big data & society*, *3*(1), 2053951715622512.

22. Jordan, M. I., & Mitchell, T. M. (2015). Machine learning: Trends, perspectives, and prospects. *Science*, *349*(6245), 255-260.

23. Amruthnath, N., & Gupta, T. (2018, April). A research study on unsupervised machine learning algorithms for early fault detection in predictive maintenance. In *2018 5th international conference on industrial engineering and applications (ICIEA)* (pp. 355-361). IEEE.

24.. Díaz-Rodríguez, N., Del Ser, J., Coeckelbergh, M., de Prado, M. L., Herrera-Viedma, E., & Herrera, F. (2023). Connecting the dots in trustworthy Artificial Intelligence: From AI principles, ethics, and key requirements to responsible AI systems and regulation. *Information Fusion*, *99*, 101896.

25. Usama, M., Qadir, J., Raza, A., Arif, H., Yau, K. L. A., Elkhatib, Y., ... & Al-Fuqaha, A. (2019). Unsupervised machine learning for networking: Techniques, applications and research challenges. *IEEE access*, *7*, 65579-65615.

26. Kerenidis, I., Landman, J., Luongo, A., & Prakash, A. (2019). q-means: A quantum algorithm for unsupervised machine learning. *Advances in neural information processing systems*, *32*.

27. Panda, Sibaram Prasad. "Augmented and Virtual Reality in Intelligent Systems." INTERNATIONAL JOURNAL OF ADVANCED MULTIDISCIPLINARY RESEARCH, CASES AND PRACTICES 1, no. 1 (2021): 23. doi:10.5281/zenodo.15401142.

ISSN 2250-1797
Issue 15 Volume 4 2025
Original Article

28. Morgan, R. K. (2012). Environmental impact assessment: the state of the art. *Impact assessment and project appraisal*, *30*(1), 5-14.

29. Canter, L. W., & Wood, C. (1996). *Environmental impact assessment* (Vol. 2). New York: McGraw-Hill.

30 Glasson, J., & Therivel, R. (2013). *Introduction to environmental impact assessment*. Routledge.

31. Newman, P. (2006). The environmental impact of cities. *Environment and Urbanization*, *18*(2), 275-295.

32. Olsen, L., Aisner, D., & McGinnis, J. M. (2007). The learning healthcare system: workshop summary.

33. Docteur, E., & Oxley, H. (2003). Health-care systems: lessons from the reform experience.

34. Panda, Sibaram. (2025). The Evolution and Defense Against Social Engineering and Phishing Attacks. International Journal of Science and Research (IJSR). 14. 397-408. 10.21275/SR25504223645.

35. Fatima, M., & Pasha, M. (2017). Survey of machine learning algorithms for disease diagnostic. *Journal of Intelligent Learning Systems and Applications*, *9*(01), 1.

36. Murali Malempati. (2022). AI Neural Network Architectures For Personalized Payment Systems: Exploring Machine Learning's Role In Real-Time Consumer Insights. Migration Letters, 19(S8), 1934–1948. Retrieved from https://migrationletters.com/index.php/ml/article/view/11632

37. Panda, Sibaram Prasad. "Exploration of End to End Big Data Engineering and Analytics." INTERNATIONAL JOURNAL OF ADVANCED MULTIDISCIPLINARY RESEARCH, CASES AND PRACTICES, 2021. doi:10.5281/zenodo.15448162.

38. Panda, S. P. (2022). CI/CD for Microservices with Azure Kubernetes Service (AKS) and Azure DevOps. International Journal of Advanced Multidisciplinary Research, Cases and Practices. https://doi.org/10.5281/zenodo.15400629

39. Ganesan, P., & Sanodia, G. (2023). Smart Infrastructure Management: Integrating AI with DevOps for Cloud-Native Applications. Journal of Artificial Intelligence & Cloud Computing. SRC/JAICC-E163. DOI: doi. org/10.47363/JAICC/2023 (2) E163 J Arti Inte & Cloud Comp, 2(1), 2-4

40. Ganesan, P. (2023). Revolutionizing Robotics with AI. Machine Learning, and Deep Learning: A Deep Dive into Current Trends and Challenges. J Artif Intell Mach Learn & Data Sci, 1(4), 1124-1128.

41. Murali Malempati, Dr. P.R. Sudha Rani. (2023). Autonomous AI Ecosystems for Seamless Digital Transactions: Exploring Neural Network-Enhanced Predictive Payment Models. International Journal of Finance (IJFIN), 36(6), 47–69.

ISSN 2250-1797
Issue 15 Volume 4 2025
Original Article

42. Panda, S. P. (2025). Artificial Intelligence Across Borders: Transforming Industries Through Intelligent Innovation. Deep Science Publishing. https://doi.org/10.70593/978-93-49910-25-6

43. 20. Alzubi, J., Nayyar, A., & Kumar, A. (2018, November). Machine learning from theory to algorithms: an overview. In *Journal of physics: conference series* (Vol. 1142, p. 012012). IOP Publishing.

44. Pamisetty, V. (2024). AI Powered Decision Support Systems in Government Financial Management: Transforming Policy Implementation and Fiscal Responsibility.

45. Muppala, M. (2025). Artificial Intelligence and Machine Learning in Maritime Shipping. International Journal of Science and Research (IJSR) . https://doi.org/10.21275/SR25605122605